

Lab 06 – Design Patterns

Group Project Report

WebTourGuide

Tour Booking & Guide Platform

Item	Details
Group ID	Y2-S1-MLB-B2G2-03
Project	WebTourGuide
Patterns applied	Strategy, State, Observer
Technology	Java 17, Spring Boot, React
Submission date	[Insert date]

Contents

1. Group Details	2
2. Design Pattern(s) Used.....	3
2.1 Pattern Roles in Our Code.....	3
2.2 Pattern Summaries	4
3. Justification for Each Pattern	5
3.1 Strategy – Destination Search (Destination Management)	5
3.2 Strategy – Tour Package Sorting (Tour Package Management)	5
3.3 Strategy – Tour Guide Ranking (Tour Guide Management)	5
3.4 Strategy – Trip Plan Export (Trip Planning Management)	6
3.5 State – Booking Lifecycle (Booking Management)	6
3.6 Observer – Support Ticket Status Changes (Customer Support Management)	7
3.7 Overall Benefits.....	8
4. Screenshots of Implementation.....	9
4.1 Strategy – Destination Search.....	9
4.2 Strategy – Tour Package Sorting	11
4.3 Strategy – Tour Guide Ranking	13
4.4 Strategy – Trip Plan Export.....	15
4.5 State – Booking Lifecycle	18
4.6 Observer – Support Ticket Status Changes.....	20
5. Code Files	22
5.1 Folder Structure	22
5.2 Class Roles.....	22
5.3 How to Run in NetBeans	23
5.4 Where the Patterns Live in the Web Application	23

1. Group Details

Group ID: Y2-S1-MLB-B2G2-03 **Project:** WebTourGuide – Tour Booking & Guide Platform

WebTourGuide is a web application where tourists browse destinations and tour packages, find and book tour guides, plan multi-day trips and raise support requests, while staff and administrators manage the catalogue, bookings and support queue. Each group member owned one module, and each module applies a design pattern (Section 2).

No.	Student Name	Student ID	Module Responsibility
1	[Member name]	[IT number]	Destination Management
2	[Member name]	[IT number]	Tour Package Management
3	[Member name]	[IT number]	Tour Guide Management
4	[Member name]	[IT number]	Booking Management
5	[Member name]	[IT number]	Trip Planning Management
6	[Member name]	[IT number]	Customer Support Management

2. Design Pattern(s) Used

Our team applied **three behavioural design patterns** across the six modules of WebTourGuide. The **Strategy** pattern is used four times, because four modules each need to choose between several interchangeable algorithms at runtime. The **State** pattern manages the booking lifecycle, and the **Observer** pattern connects Customer Support to Booking Management.

Pattern	Module	Applied to	Selected at runtime by
Strategy	Destination Management	Searching destinations by name, location, category or any field	GET <code>/api/destinations/search?by=</code>
Strategy	Tour Package Management	Sorting packages by price (low/high), duration or newest	GET <code>/api/packages?sort=</code>
Strategy	Tour Guide Management	Ranking guides by rating, experience or number of languages	GET <code>/api/guides/ranked?by=</code>
Strategy	Trip Planning Management	Exporting a trip plan as plain text, CSV or iCalendar (.ics)	GET <code>/api/trip-plans/{id}/export?format=</code>
State	Booking Management	Booking lifecycle: Pending, Confirmed, Rescheduled, Completed, Cancelled	The booking's current status
Observer	Customer Support Management	Reacting to support-ticket status changes (audit log, auto-cancel booking)	Every ticket status update

2.1 Pattern Roles in Our Code

Each pattern is implemented with the standard roles. The class names below are the same in the WebTourGuide web application and in the standalone demo project submitted with this report.

Use	Interface	Context / Subject	Concrete classes
Destination search	DestinationSearchStrategy	DestinationSearchContext	NameSearchStrategy, LocationSearchStrategy, CategorySearchStrategy, AnyFieldSearchStrategy
Package sorting	PackageSortStrategy	PackageSortContext	PriceLowToHighSort, PriceHighToLowSort, ShortestDurationSort, NewestFirstSort
Guide ranking	GuideRankingStrategy	GuideRankingContext	RatingRankingStrategy, ExperienceRankingStrategy, LanguageCountRankingStrategy
Trip plan export	TripPlanExportStrategy	TripPlanExporter	PlainTextExportStrategy, CsvExportStrategy, CalendarExportStrategy
Booking lifecycle	BookingState	Booking	PendingState, ConfirmedState, RescheduledState, CompletedState, CancelledState
Ticket status	TicketStatusObserver	SupportTicketService (subject)	TicketAuditLogObserver, BookingCancellationObserver

2.2 Pattern Summaries

Strategy defines a family of algorithms, puts each one in its own class behind a common interface, and lets a context object use any of them interchangeably. The algorithm can be swapped at runtime without changing the context.

State lets an object change its behaviour when its internal state changes. Each state is a class that decides which actions are allowed and what the next state is, replacing large conditional statements on a status field.

Observer defines a one-to-many dependency: when the subject changes, all registered observers are notified automatically. The subject only knows the observer interface, not what each observer does.

3. Justification for Each Pattern

For each use we describe the problem in WebTourGuide, why the chosen pattern fits it, and how it improved the design compared with the straightforward alternative.

3.1 Strategy – Destination Search (Destination Management)

Problem. Tourists search destinations in four different ways: by name, by location, by exact category, or by a keyword that may appear in any of those fields. A first version would put an `if/else` chain on the search type inside `DestinationService`, and every new search mode would mean editing and re-testing that method.

Why Strategy fits. The four searches are different algorithms for the same task ("find destinations for a keyword") and the user chooses one at runtime through the `by` parameter. This is exactly the situation the Strategy pattern targets.

How it improves the design.

- **Open/Closed:** a new search mode (for example, search by nearby coordinates) is a new class; `DestinationService` and the controller do not change.
- **No conditional logic:** the service looks up the strategy by its key and calls `executeSearch()`; there is no `switch` on the search type.
- **Single Responsibility and testability:** each strategy contains one query and is unit-tested on its own (`DestinationSearchStrategyTest`).

3.2 Strategy – Tour Package Sorting (Tour Package Management)

Problem. The package list can be ordered by lowest price, highest price, shortest duration or newest first. Hard-coding these orders in the service would mix presentation choices with business logic and make the list endpoint grow with every new ordering.

Why Strategy fits. Each ordering is an interchangeable algorithm, represented as a `Comparator`. The tourist selects it with `?sort=` and the `PackageSortContext` applies whichever strategy it holds.

How it improves the design.

- New orderings (for example, "most popular") are added as one small class each.
- The context returns a new sorted list and never modifies the input, so the same data can be sorted several ways safely.
- Unknown sort keys are rejected with a clear error listing the allowed values, generated from the registered strategies.

3.3 Strategy – Tour Guide Ranking (Tour Guide Management)

Problem. Tourists want to see the "best" guides first, but "best" depends on what they care about: highest rating, most experience, or the most languages spoken. Each definition needs its own tie-breaking rules.

Why Strategy fits. Each ranking rule is a separate, interchangeable algorithm with its own tie-breakers. The `GuideRankingContext` ranks the guide list with whichever rule the tourist chose through `/api/guides/ranked?by=`.

How it improves the design.

- Ranking rules are isolated, so changing how experience ties are broken cannot affect the rating ranking.
- The same ranking can be combined with filters (for example, available guides only) without new code paths.
- Each rule is verified independently in `GuideRankingStrategyTest`.

3.4 Strategy – Trip Plan Export (Trip Planning Management)

Problem. A tourist can download their itinerary as a readable text file, as a CSV spreadsheet, or as an iCalendar (.ics) file that imports into Google Calendar or Outlook. The formats share no output logic and each needs its own file extension and content type.

Why Strategy fits. All three formats perform the same operation ("turn this trip plan into a file") in completely different ways, chosen by the user at download time.

How it improves the design.

- Each format, including its MIME type and file extension, is encapsulated in one class, so the controller simply asks the strategy for them.
- Adding a PDF export later requires only a new `TripPlanExportStrategy` implementation.
- Format-specific rules (CSV quoting, iCalendar escaping) stay out of `TripPlanService`.

3.5 State – Booking Lifecycle (Booking Management)

Problem. A booking moves through Pending, Confirmed, Rescheduled, Completed and Cancelled. Which actions are allowed depends on the current status: for example, a completed booking cannot be cancelled, and a rescheduled booking must be confirmed again. Implemented with conditionals, every action method in `BookingService` would repeat a `switch` on the status, and adding a status would mean editing all of them.

Why State fits. The booking's behaviour changes with its internal status, and the rules form a clear state machine. The State pattern models each status as its own class that allows only its valid transitions.

Current state	Allowed actions → next state	Rejected actions
Pending	confirm → Confirmed; reschedule → Rescheduled; cancel → Cancelled	complete
Confirmed	complete → Completed; reschedule → Rescheduled; cancel → Cancelled	confirm
Rescheduled	confirm → Confirmed; reschedule → Rescheduled (new date); cancel → Cancelled	complete
Completed	none (final state)	all actions
Cancelled	none (final state)	all actions

How it improves the design.

- Invalid transitions are rejected in one place: `BookingState` rejects every action by default, and each state overrides only what it allows.
- `Booking` and `BookingService` contain no status conditionals; they call `booking.confirm()`, `booking.cancel()` and so on.
- The lifecycle is easy to read and test: each state class is a few lines and maps directly to the table above (`BookingStateTest`).

3.6 Observer – Support Ticket Status Changes (Customer Support Management)

Problem. When staff change a support ticket's status, other parts of the system must react. Resolving a cancellation-request ticket must actually cancel the linked booking, and every status change should be written to an audit log. In the future we also want e-mail notifications. Putting all of this inside `SupportTicketService.updateStatus()` would tightly couple Customer Support to Booking Management and make the method grow with every new side effect.

Why Observer fits. One event (a status change) must trigger several independent reactions, and the set of reactions is expected to grow. The service acts as the subject and notifies every registered `TicketStatusObserver`.

How it improves the design.

- **Loose coupling:** the support service does not depend on booking logic; `BookingCancellationObserver` owns that rule.
- **Extensibility:** an e-mail notifier can be added as a new observer, and Spring registers it automatically, with no change to the service.
- **Consistency:** observers run before the ticket is saved, so if the booking cannot be cancelled, the ticket is not marked as resolved.

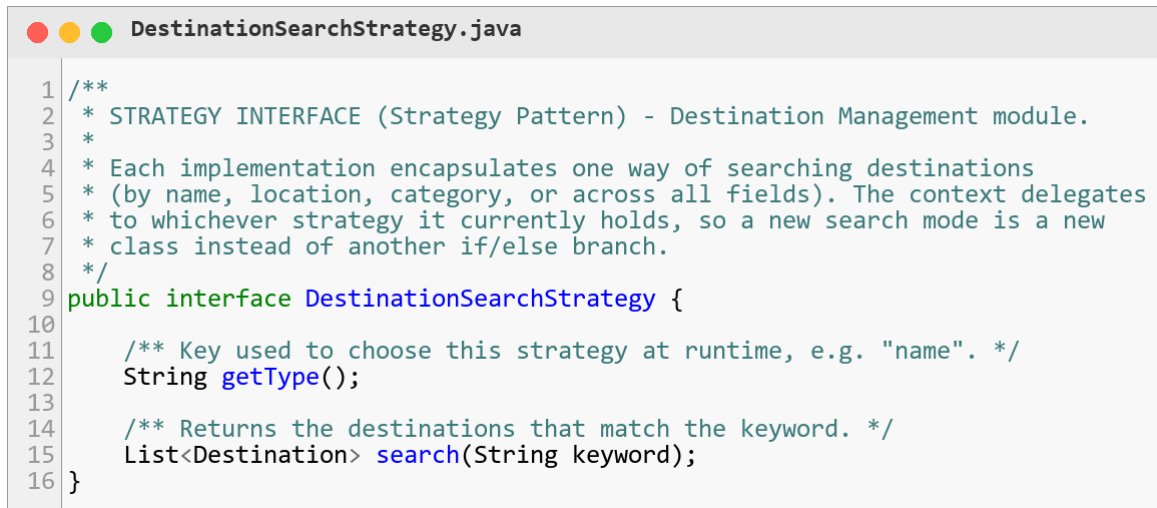
3.7 Overall Benefits

Design principle	How the patterns support it in WebTourGuide
Open/Closed Principle	New search modes, sort orders, ranking rules, export formats, booking states and ticket reactions are added as new classes without modifying existing ones.
Single Responsibility	Each strategy, state and observer class does one job; the services coordinate instead of containing every rule.
Composition over inheritance	Contexts hold a strategy reference and delegate to it, so behaviour can be swapped at runtime with <code>setStrategy()</code> .
Loose coupling	Contexts and subjects depend only on interfaces, never on concrete classes.
Testability	Small, focused classes are covered by dedicated unit tests in the web project.

4. Screenshots of Implementation

The figures below show the key classes of each pattern (interface, a concrete class and the context or subject) and the console output of the demo program `Main.java`, run from the submitted `WebTourGuidePatterns` project. Package and import lines are omitted from the code screenshots for brevity.

4.1 Strategy – Destination Search



```
1  /**
2  * STRATEGY INTERFACE (Strategy Pattern) - Destination Management module.
3  *
4  * Each implementation encapsulates one way of searching destinations
5  * (by name, location, category, or across all fields). The context delegates
6  * to whichever strategy it currently holds, so a new search mode is a new
7  * class instead of another if/else branch.
8  */
9  public interface DestinationSearchStrategy {
10
11     /** Key used to choose this strategy at runtime, e.g. "name". */
12     String getType();
13
14     /** Returns the destinations that match the keyword. */
15     List<Destination> search(String keyword);
16 }
```

Figure 1: Strategy interface `DestinationSearchStrategy`



```
1  /** CONCRETE STRATEGY: partial, case-insensitive match on the destination name. */
2  public class NameSearchStrategy implements DestinationSearchStrategy {
3
4     private final DestinationRepository repository;
5
6     public NameSearchStrategy(DestinationRepository repository) {
7         this.repository = repository;
8     }
9
10     @Override
11     public String getType() {
12         return "name";
13     }
14
15     @Override
16     public List<Destination> search(String keyword) {
17         return repository.findByNameContainingIgnoreCase(keyword);
18     }
19 }
```

Figure 2: Concrete strategy `NameSearchStrategy`

```
DestinationSearchContext.java
1 /**
2  * CONTEXT CLASS (Strategy Pattern).
3  *
4  * Holds a reference to a DestinationSearchStrategy and delegates the search
5  * to it. The strategy can be swapped at runtime with setStrategy() without
6  * changing this class (composition over inheritance).
7  */
8 public class DestinationSearchContext {
9
10     private DestinationSearchStrategy strategy;
11
12     public DestinationSearchContext(DestinationSearchStrategy strategy) {
13         this.strategy = strategy;
14     }
15
16     /** Switches the search behaviour at runtime. */
17     public void setStrategy(DestinationSearchStrategy strategy) {
18         this.strategy = strategy;
19     }
20
21     public DestinationSearchStrategy getStrategy() {
22         return strategy;
23     }
24
25     public List<Destination> executeSearch(String keyword) {
26         return strategy.search(keyword);
27     }
28 }
```

Figure 3: Context class DestinationSearchContext with setStrategy()

```
Main.java
1 /** 1. Strategy Pattern - the same context switches search strategy at runtime. */
2 private static void demoDestinationSearch(List<Destination> destinations) {
3     heading("1. STRATEGY - Destination search");
4     DestinationRepository repository = new DestinationRepository(destinations);
5
6     DestinationSearchContext context =
7         new DestinationSearchContext(new NameSearchStrategy(repository));
8     printSearch(context, "rock");
9
10    // switch the strategy at runtime - same context, new behaviour
11    context.setStrategy(new LocationSearchStrategy(repository));
12    printSearch(context, "galle");
13
14    context.setStrategy(new CategorySearchStrategy(repository));
15    printSearch(context, "historical");
16
17    context.setStrategy(new AnyFieldSearchStrategy(repository));
18    printSearch(context, "beach");
19 }
```

Figure 4: Main switching the search strategy at runtime

```

Output - WebTourGuidePatterns (run)

=====
1. STRATEGY - Destination search
=====

Search by 'name' for "rock":
  - Sigiriya Rock (Historical, Dambulla)
  - Ella Rock (Mountain, Badulla)
Search by 'location' for "galle":
  - Unawatuna Bay (Beach, Galle)
Search by 'category' for "historical":
  - Sigiriya Rock (Historical, Dambulla)
  - Temple of the Tooth (Historical, Kandy)
Search by 'any' for "beach":
  - Mirissa Beach (Beach, Matara)
  - Unawatuna Bay (Beach, Galle)

```

Figure 5: Output: the same context searching with four different strategies

4.2 Strategy – Tour Package Sorting

```

PackageSortStrategy.java

1 /**
2  * STRATEGY INTERFACE (Strategy Pattern) - Tour Package Management module.
3  *
4  * Each implementation defines one ordering of tour packages. Tourists pick the
5  * order at runtime (GET /api/packages?sort=...); a new ordering is a new class.
6  */
7 public interface PackageSortStrategy {
8
9     /** Key used to choose this strategy, e.g. "price-asc". */
10    String getType();
11
12    Comparator<TourPackage> comparator();
13 }

```

Figure 6: Strategy interface PackageSortStrategy

```

PriceLowToHighSort.java

1 /** CONCRETE STRATEGY: cheapest package first. */
2 public class PriceLowToHighSort implements PackageSortStrategy {
3
4     @Override
5     public String getType() {
6         return "price-asc";
7     }
8
9     @Override
10    public Comparator<TourPackage> comparator() {
11        return Comparator.comparing(TourPackage::getPrice);
12    }
13 }

```

Figure 7: Concrete strategy PriceLowToHighSort

```

PackageSortContext.java
1  /**
2   * CONTEXT CLASS (Strategy Pattern): sorts packages with whichever
3   * strategy it currently holds.
4   */
5  public class PackageSortContext {
6
7      private PackageSortStrategy strategy;
8
9      public PackageSortContext(PackageSortStrategy strategy) {
10         this.strategy = strategy;
11     }
12
13     /** Switches the sort order at runtime. */
14     public void setStrategy(PackageSortStrategy strategy) {
15         this.strategy = strategy;
16     }
17
18     /** Returns a new sorted list; the input list is not modified. */
19     public List<TourPackage> sort(List<TourPackage> packages) {
20         return packages.stream().sorted(strategy.comparator()).toList();
21     }
22 }

```

Figure 8: Context class PackageSortContext

```

Output - WebTourGuidePatterns (run)

=====
2. STRATEGY - Tour package sorting
=====

Sorted by 'price-asc':
    Kandy City Day Trip      1 days $   75.00 added 2026-09-30
    Ella Hiking Weekend     2 days $  150.00 added 2026-07-20
    Southern Coast Escape    4 days $  320.00 added 2026-08-01
    Cultural Triangle Tour    6 days $  540.00 added 2026-09-12
Sorted by 'price-desc':
    Cultural Triangle Tour    6 days $  540.00 added 2026-09-12
    Southern Coast Escape    4 days $  320.00 added 2026-08-01
    Ella Hiking Weekend     2 days $  150.00 added 2026-07-20
    Kandy City Day Trip      1 days $   75.00 added 2026-09-30
Sorted by 'duration':
    Kandy City Day Trip      1 days $   75.00 added 2026-09-30
    Ella Hiking Weekend     2 days $  150.00 added 2026-07-20
    Southern Coast Escape    4 days $  320.00 added 2026-08-01
    Cultural Triangle Tour    6 days $  540.00 added 2026-09-12
Sorted by 'newest':
    Kandy City Day Trip      1 days $   75.00 added 2026-09-30
    Cultural Triangle Tour    6 days $  540.00 added 2026-09-12
    Southern Coast Escape    4 days $  320.00 added 2026-08-01
    Ella Hiking Weekend     2 days $  150.00 added 2026-07-20

```

Figure 9: Output: one package list sorted by four strategies

4.3 Strategy – Tour Guide Ranking

```
GuideRankingStrategy.java
1 /**
2  * STRATEGY INTERFACE (Strategy Pattern) - Tour Guide Management module.
3  *
4  * Each implementation defines one way of ranking guides best-first
5  * (rating, experience, languages). Chosen at runtime via
6  * GET /api/guides/ranked?by=...
7  */
8 public interface GuideRankingStrategy {
9
10     /** Key used to choose this strategy, e.g. "rating". */
11     String getType();
12
13     /** Orders guides best-first. */
14     Comparator<TourGuide> comparator();
15 }
```

Figure 10: Strategy interface GuideRankingStrategy

```
LanguageCountRankingStrategy.java
1 /** CONCRETE STRATEGY: guides who speak the most languages first. */
2 public class LanguageCountRankingStrategy implements GuideRankingStrategy {
3
4     @Override
5     public String getType() {
6         return "languages";
7     }
8
9     @Override
10    public Comparator<TourGuide> comparator() {
11        return Comparator
12            .comparingLong(LanguageCountRankingStrategy::languageCount)
13            .reversed();
14    }
15
16    /** Counts the entries in the comma-separated languages field. */
17    static long languageCount(TourGuide guide) {
18        if (guide.getLanguages() == null) return 0;
19        return Arrays.stream(guide.getLanguages().split(","))
20            .filter(lang -> !lang.isBlank())
21            .count();
22    }
23 }
```

Figure 11: Concrete strategy LanguageCountRankingStrategy

```

GuideRankingContext.java
1 /**
2  * CONTEXT CLASS (Strategy Pattern): ranks guides with whichever
3  * strategy it currently holds.
4  */
5 public class GuideRankingContext {
6
7     private GuideRankingStrategy strategy;
8
9     public GuideRankingContext(GuideRankingStrategy strategy) {
10         this.strategy = strategy;
11     }
12
13     /** Switches the ranking rule at runtime. */
14     public void setStrategy(GuideRankingStrategy strategy) {
15         this.strategy = strategy;
16     }
17
18     /** Returns a new ranked list; the input list is not modified. */
19     public List<TourGuide> rank(List<TourGuide> guides) {
20         return guides.stream().sorted(strategy.comparator()).toList();
21     }
22 }

```

Figure 12: Context class GuideRankingContext

```

Output - WebTourGuidePatterns (run)

=====
3. STRATEGY - Tour guide ranking
=====

Ranked by 'rating':
  1. Amara Fernando   rating 4.8   7 yrs   [English, Sinhala, Tamil]
  2. Kasun Perera     rating 4.5   4 yrs   [English, Sinhala]
  3. Ruwan Jayasuriya rating 4.4   5 yrs   [English, German, French, Sinhala]
  4. Nimal Silva      rating 4.2   2 yrs   [English, Tamil]

Ranked by 'experience':
  1. Amara Fernando   rating 4.8   7 yrs   [English, Sinhala, Tamil]
  2. Ruwan Jayasuriya rating 4.4   5 yrs   [English, German, French, Sinhala]
  3. Kasun Perera     rating 4.5   4 yrs   [English, Sinhala]
  4. Nimal Silva      rating 4.2   2 yrs   [English, Tamil]

Ranked by 'languages':
  1. Ruwan Jayasuriya rating 4.4   5 yrs   [English, German, French, Sinhala]
  2. Amara Fernando   rating 4.8   7 yrs   [English, Sinhala, Tamil]
  3. Kasun Perera     rating 4.5   4 yrs   [English, Sinhala]
  4. Nimal Silva      rating 4.2   2 yrs   [English, Tamil]

```

Figure 13: Output: the same guides ranked by rating, experience and languages

4.4 Strategy – Trip Plan Export

```

1  /**
2  * STRATEGY INTERFACE (Strategy Pattern) - Trip Planning Management module.
3  *
4  * Each implementation writes the same itinerary in a different file format
5  * (plain text, CSV, iCalendar). The tourist picks the format at runtime with
6  * GET /api/trip-plans/{id}/export?format=...
7  */
8  public interface TripPlanExportStrategy {
9
10     /** Key used to choose this strategy, e.g. "csv". */
11     String getFormat();
12
13     /** File extension without the dot, e.g. "csv". */
14     String getFileExtension();
15
16     /** Renders the plan as file content. */
17     String export(TripPlan plan);
18 }

```

Figure 14: Strategy interface *TripPlanExportStrategy*

```

1  /** CONCRETE STRATEGY: one row per day, for opening in Excel or Google Sheets. */
2  public class CsvExportStrategy implements TripPlanExportStrategy {
3
4      @Override
5      public String getFormat() {
6          return "csv";
7      }
8
9      @Override
10     public String getFileExtension() {
11         return "csv";
12     }
13
14     @Override
15     public String export(TripPlan plan) {
16         StringBuilder out = new StringBuilder();
17         out.append("Day, Destination, Accommodation, Activities\n");
18         for (TripPlanItem item : plan.getItems()) {
19             Destination place = item.getDestination();
20             out.append(item.getDayNumber()).append(',')
21                 .append(cell(place != null ? place.getName() : null)).append(',')
22                 .append(cell(item.getAccommodation())).append(',')
23                 .append(cell(item.getActivities())).append('\n');
24         }
25         return out.toString();
26     }
27
28     /** Quotes a value and doubles inner quotes, so commas and quotes survive. */
29     private String cell(String value) {
30         if (value == null) return "";
31         return "'" + value.replace("\\", "\\") + "'";
32     }
33 }

```

Figure 15: Concrete strategy *CsvExportStrategy*



```
1 /**
2  * CONTEXT CLASS (Strategy Pattern): exports a trip plan with whichever
3  * format strategy it currently holds.
4  */
5 public class TripPlanExporter {
6
7     private TripPlanExportStrategy strategy;
8
9     public TripPlanExporter(TripPlanExportStrategy strategy) {
10         this.strategy = strategy;
11     }
12
13     /** Switches the export format at runtime. */
14     public void setStrategy(TripPlanExportStrategy strategy) {
15         this.strategy = strategy;
16     }
17
18     public String fileName(TripPlan plan) {
19         return "trip-plan-" + plan.getId() + "." + strategy.getFileExtension();
20     }
21
22     public String export(TripPlan plan) {
23         return strategy.export(plan);
24     }
25 }
```

Figure 16: Context class TripPlanExporter

```
Output - WebTourGuidePatterns (run)

=====
4. STRATEGY - Trip plan export
=====

--- trip-plan-7.txt (format 'text') ---
Hill Country Weekend
2026-12-05 to 2026-12-07
Day 1: Temple of the Tooth | Stay: Kandy Guest House
Day 2: Ella Rock | Stay: Ella Flower Garden Resort
Day 3: Rest day
--- trip-plan-7.csv (format 'csv') ---
Day,Destination,Accommodation,Activities
1,"Temple of the Tooth","Kandy Guest House","Temple visit"
2,"Ella Rock","Ella Flower Garden Resort","Hiking, Nine Arch Bridge"
3,,,
--- trip-plan-7.ics (format 'ics') ---
BEGIN:VCALENDAR
VERSION:2.0
BEGIN:VEVENT
DTSTART;VALUE=DATE:20261205
SUMMARY:Hill Country Weekend - Day 1: Temple of the Tooth
END:VEVENT
BEGIN:VEVENT
DTSTART;VALUE=DATE:20261206
SUMMARY:Hill Country Weekend - Day 2: Ella Rock
END:VEVENT
BEGIN:VEVENT
DTSTART;VALUE=DATE:20261207
SUMMARY:Hill Country Weekend - Day 3: Rest day
END:VEVENT
END:VCALENDAR
```

Figure 17: Output: one trip plan exported as text, CSV and iCalendar

4.5 State – Booking Lifecycle

```

1  /**
2  * STATE INTERFACE (State Pattern) - Booking Management module.
3  *
4  * Each booking status is a class that decides which actions are allowed from
5  * it and which status comes next. By default every action is rejected; each
6  * concrete state overrides only the transitions it allows, so the service no
7  * longer needs an if/else per status.
8  */
9  public interface BookingState {
10
11     BookingStatus getStatus();
12
13     default void confirm(Booking booking) {
14         throw notAllowed("confirm");
15     }
16
17     default void complete(Booking booking) {
18         throw notAllowed("complete");
19     }
20
21     default void cancel(Booking booking) {
22         throw notAllowed("cancel");
23     }
24
25     default void reschedule(Booking booking, LocalDate newDate) {
26         throw notAllowed("reschedule");
27     }
28
29     private IllegalStateException notAllowed(String action) {
30         String status = getStatus().name().toLowerCase();
31         return new IllegalStateException(
32             "Cannot " + action + " a " + status + " booking");
33     }
34 }

```

Figure 18: State interface BookingState with default rejections

```

1  /** CONCRETE STATE: a new booking waiting for staff confirmation. */
2  public class PendingState implements BookingState {
3
4     @Override
5     public BookingStatus getStatus() {
6         return BookingStatus.PENDING;
7     }
8
9     @Override
10    public void confirm(Booking booking) {
11        booking.setStatus(BookingStatus.CONFIRMED);
12    }
13
14    @Override
15    public void cancel(Booking booking) {
16        booking.setStatus(BookingStatus.CANCELLED);
17    }
18
19    @Override
20    public void reschedule(Booking booking, LocalDate newDate) {
21        booking.setBookingDate(newDate);
22        booking.setStatus(BookingStatus.RESCHEDULED);
23    }
24 }

```

Figure 19: Concrete state PendingState

```

1  /**
2  * CONTEXT CLASS (State Pattern).
3  *
4  * A booking delegates every lifecycle action to the state object for its
5  * current status. The state decides whether the action is allowed and what
6  * the next status is, so Booking itself contains no status if/else logic.
7  */
8  public class Booking {
9
10     private final Long id;
11     private final String packageTitle;
12     private LocalDate bookingDate;
13     private BookingStatus status = BookingStatus.PENDING;
14
15     public Booking(Long id, String packageTitle, LocalDate bookingDate) {
16         this.id = id;
17         this.packageTitle = packageTitle;
18         this.bookingDate = bookingDate;
19     }
20
21     /** The state object for the current status; it decides what is allowed. */
22     public BookingState currentState() {
23         return BookingStateFactory.of(status);
24     }
25
26     public void confirm() { currentState().confirm(this); }
27     public void complete() { currentState().complete(this); }
28     public void cancel() { currentState().cancel(this); }
29
30     public void reschedule(LocalDate newDate) {
31         currentState().reschedule(this, newDate);
32     }
33
34     // getters, package-private setters and toString() omitted
35 }

```

Figure 20: Context class Booking delegating to its current state

```

Output - WebTourGuidePatterns (run)

=====
5. STATE - Booking lifecycle
=====
Created:      Booking #101 [Southern Coast Escape, 2026-11-10] status=PENDING
confirm():    Booking #101 [Southern Coast Escape, 2026-11-10] status=CONFIRMED
reschedule(): Booking #101 [Southern Coast Escape, 2026-11-20] status=RESCHEDULED
confirm():    Booking #101 [Southern Coast Escape, 2026-11-20] status=CONFIRMED
complete():   Booking #101 [Southern Coast Escape, 2026-11-20] status=COMPLETED
cancel():     rejected -> Cannot cancel a completed booking

```

Figure 21: Output: valid transitions succeed; cancelling a completed booking is rejected

4.6 Observer – Support Ticket Status Changes

```
1  /**
2  * OBSERVER INTERFACE (Observer Pattern) - Customer Support Management module.
3  *
4  * SupportTicketService (the subject) notifies every registered observer when
5  * a ticket's status changes. Each observer reacts in its own way, so new side
6  * effects (emails, audit logs, booking updates) can be added without editing
7  * the service.
8  */
9  public interface TicketStatusObserver {
10
11      void onStatusChanged(SupportTicket ticket, TicketStatus previousStatus,
12                          String changedBy);
13  }
```

Figure 22: Observer interface TicketStatusObserver

```
1  /**
2  * SUBJECT (Observer Pattern).
3  *
4  * Keeps a list of observers and notifies all of them after a ticket's status
5  * changes. The service does not know what each observer does - it only calls
6  * the shared onStatusChanged() method.
7  */
8  public class SupportTicketService {
9
10     private final List<TicketStatusObserver> observers = new ArrayList<>();
11
12     public void addObserver(TicketStatusObserver observer) {
13         observers.add(observer);
14     }
15
16     public void removeObserver(TicketStatusObserver observer) {
17         observers.remove(observer);
18     }
19
20     /** Staff/admin changes a ticket's status; every observer is notified. */
21     public void updateStatus(SupportTicket ticket, TicketStatus newStatus,
22                             String changedBy) {
23         TicketStatus previousStatus = ticket.getStatus();
24         ticket.setStatus(newStatus);
25         notifyObservers(ticket, previousStatus, changedBy);
26     }
27
28     private void notifyObservers(SupportTicket ticket, TicketStatus previousStatus,
29                                 String changedBy) {
30         for (TicketStatusObserver observer : observers) {
31             observer.onStatusChanged(ticket, previousStatus, changedBy);
32         }
33     }
34 }
```

Figure 23: Subject SupportTicketService notifying registered observers

```

1  /**
2  * CONCRETE OBSERVER: resolving a cancellation request actually cancels the
3  * linked booking, so Booking Management stays in sync with Customer Support.
4  */
5  public class BookingCancellationObserver implements TicketStatusObserver {
6
7      @Override
8      public void onStatusChanged(SupportTicket ticket, TicketStatus previousStatus,
9                                String changedBy) {
10         if (ticket.getStatus() != TicketStatus.RESOLVED
11             || ticket.getType() != TicketType.CANCELLATION_REQUEST
12             || ticket.getBooking() == null) {
13             return;
14         }
15         Booking booking = ticket.getBooking();
16         // A cancelled or completed booking is final, so there is nothing to cancel.
17         if (booking.getStatus() == BookingStatus.CANCELLED
18             || booking.getStatus() == BookingStatus.COMPLETED) {
19             return;
20         }
21         booking.cancel();
22         System.out.println("    [BookingCancellation] Booking #" + booking.getId()
23                             + " is now " + booking.getStatus());
24     }
25 }

```

Figure 24: Concrete observer *BookingCancellationObserver*

```

Output - WebTourGuidePatterns (run)

=====
6. OBSERVER - Support ticket status changes
=====

Before: Booking #202 [Ella Hiking Weekend, 2026-12-01] status=PENDING
Staff moves ticket #55 to IN_PROGRESS:
    [AuditLog] Ticket #55 changed OPEN -> IN_PROGRESS by 'staff@webtourguide.com'
Staff moves ticket #55 to RESOLVED:
    [AuditLog] Ticket #55 changed IN_PROGRESS -> RESOLVED by 'staff@webtourguide.com'
    [BookingCancellation] Booking #202 is now CANCELLED
After:  Booking #202 [Ella Hiking Weekend, 2026-12-01] status=CANCELLED

```

Figure 25: Output: resolving a cancellation request notifies both observers and cancels the booking

5. Code Files

The submitted source code is a clean, self-contained Java 17 project, **WebTourGuidePatterns**, that runs without a database or web server. It reproduces the pattern classes of the WebTourGuide web application using sample data, so the patterns can be run and inspected directly in NetBeans. Each package corresponds to one module, and every class is labelled in its Javadoc with its pattern role (STRATEGY INTERFACE, CONCRETE STRATEGY, CONTEXT CLASS, STATE INTERFACE, CONCRETE STATE, SUBJECT, OBSERVER INTERFACE, CONCRETE OBSERVER).

5.1 Folder Structure

```

Lab06_DesignPatterns/
├── Lab06_DesignPatterns_Report_WebTourGuide.docx    (this report)
├── README.md                                         (how to run)
├── screenshots/                                     (figures used in this report)
└── WebTourGuidePatterns/                           (NetBeans / Maven project)
    ├── pom.xml
    └── src/main/java/com/webtourguide/patterns/
        ├── Main.java                               Main class - runs all six demos
        ├── model/                                   Sample data classes (Destination, TourPackage, ...)
        ├── destination/                             Strategy: destination search
        ├── tourpackage/                             Strategy: package sorting
        ├── tourguide/                               Strategy: guide ranking
        ├── tripplan/                                Strategy: trip plan export
        ├── booking/                                 State: booking lifecycle
        └── support/                                 Observer: support ticket status
  
```

5.2 Class Roles

Role	Classes
Main class	Main – creates sample data and demonstrates every pattern, including switching strategies at runtime.
Interfaces	DestinationSearchStrategy , PackageSortStrategy , GuideRankingStrategy , TripPlanExportStrategy , BookingState , TicketStatusObserver
Context classes	DestinationSearchContext , PackageSortContext , GuideRankingContext , TripPlanExporter , Booking (State context), SupportTicketService (Observer subject)
Concrete classes	4 search strategies, 4 sort strategies, 3 ranking strategies, 3 export strategies, 5 booking states, 2 observers
Supporting classes	Model classes in model/ , DestinationRepository (in-memory data), BookingStateFactory , enums BookingStatus , TicketStatus , TicketType

5.3 How to Run in NetBeans

1. Open NetBeans and choose **File → Open Project**.
2. Select the `WebTourGuidePatterns` folder (NetBeans recognises it as a Maven project) and click **Open Project**.
3. Right-click the project and choose **Run** (or open `Main.java` and press **Shift + F6**).
4. The Output window shows the six demonstrations seen in Section 4.

Without an IDE, the project can be run from its folder with `mvn compile exec:java`, or compiled with `javac` and run with `java com.webtourguide.patterns.Main`.

5.4 Where the Patterns Live in the Web Application

In the full WebTourGuide application (Spring Boot API), the same patterns are wired in as Spring components and selected through the REST API:

Pattern use	Package in webtourguide-api
Destination search	<code>com.webtourguide.destination.search</code>
Package sorting	<code>com.webtourguide.tourpackage.sorting</code>
Guide ranking	<code>com.webtourguide.tourguide.ranking</code>
Trip plan export	<code>com.webtourguide.tripplan.export</code>
Booking lifecycle	<code>com.webtourguide.booking.state</code>
Ticket status observers	<code>com.webtourguide.support.observer</code>